FMI Layered Standard for DAE (FMI-LS-DAE)

Contents

1. Introduction . . . 2
   1.1. Intent of this Document . . . 2
   1.2. How to read this Document . . . 2
   1.3. Overview . . . 2
2. Common Concepts . . . 2
   Mathematical Notation . . . 3
      DAE representation . . . 3
         Fully-Implicit DAE . . . 3
         Semi-Explicit Index 1 / Hessenberg index 1 DAE . . . 3
         Relationship between fully-implicit and semi-explicit DAE . . . 3
         Semi-Explicit DAE . . . 4
         Mass matrix DAE . . . 4
         Linear time-invariant DAE . . . 4
      Indices . . . 4
         Differential Index . . . 4
         Perturbation Index . . . 4
            Example of Structural Singularity . . . 5
      Hello World DAE Example . . . 7
         Hello World DAE - Fully-Implicit DAE . . . 7
         Hello World DAE - Semi-Explicit DAE . . . 7
         Hello World DAE - Explicit ODE . . . 7
3. Use Cases . . . 8
   3.1. Generic DAE Use Case . . . 8
      3.1.1. Modelica Example CauerLowPassAnalog . . . 8
      3.1.2. Scaling Power Grids Example . . . 10
4. Layered Standard Manifest File . . . 11
   4.1. Algebraic variables . . . 12
   4.2. Model Structure . . . 13
      4.2.1. Formulation . . . 16
References . . . 19

This layered standard on top of FMI 3.0 defines how to exchange dynamic models in differential algebraic equation form.

> Although the document refers to version 3.0 of the FMI standard, everything described in this document also applies to all subsequent minor versions. For further information on compatibility, see section Versioning and Layered Standards in the FMI 3.0 specification.

Copyright © 2025 The Modelica Association Project FMI.

This document is licensed under the Attribution-ShareAlike 4.0 International license. The code is released under the 2-Clause BSD License. The license text can be found in the LICENSE.txt file that accompanies this distribution.

1. Introduction

1.1. Intent of this Document

A differential-algebraic system of equations (DAE) is a system of equations that either contains differential equations and algebraic equations, or is equivalent to such a system [1].

- Avoid the requirement of index reduction inside of FMUs.
  - This may improve accuracy due to better drift handling.
- Avoid local nonlinear equation solvers inside of FMUs.
  - This may improve accuracy and avoid problems with different local and global error tolerances.
- Preserve the sparseness of DAE systems which is lost for the corresponding reduced ODE systems.
  - This may improve the performance by usage of the sparseness.
- Allow connections between constraint FMUs.
  - Connecting reduced ODE FMUs may lead globally to a non-solvable (singular) system but not for unreduced DAE FMUs.

1.2. How to read this Document

1.3. Overview

2. Common Concepts

Mathematical Notation

Table 1. Additional symbols for specific variable types.

Variable	Description
$\mathbf{z}_c$	A vector of real continuous-time variables representing the continuous-time algebraic variables.

DAE representation

There are different forms of DAEs.

For simplicity discontinuous states are currently disregarded.

Fully-Implicit DAE

The generic *fully-implicit DAE*, or *implicit DAE*, formulation:

$$F(\dot{z}(t),\ z(t),\ t) = 0$$

where $z(t)$ represents state and additional variables and t is time.

A similar and sometimes more useful representation:

$$F(\dot{x}(t),\ x(t),\ y(t),\ t) = 0$$

with dynamic variables $x(t)$, algebraic variables $y(t)$ and time t.

Note that $x(t)$ is not necessarily the state variable, more on this in section 2.1.1.

Semi-Explicit Index 1 / Hessenberg index 1 DAE

The *semi-explicit index 1*, also called *Hessenberg index 1*, DAE representation

$$\begin{aligned} \dot{x}(t) &= f(x(t),\ y(t),\ t) \\ 0 &= g(x(t),\ y(t),\ t) \end{aligned}$$

with g solvable for y, $det\left(\frac{\partial}{\partial y} g\right) \neq 0$.

Here f is called the *differential equations* and g the *algebraic equations*.

Relationship between fully-implicit and semi-explicit DAE

A fully implicit DAE

$$F(\dot{x}(t),\ x(t),\ t) = 0$$

can always be rewritten as a semi-explicit DAE by introducing a new algebraic variable x'

$$\begin{aligned} \dot{x}(t) &= x'(t) \\ 0 &= F(x'(t),\ x(t),\ t) \end{aligned}$$

Semi-Explicit DAE

The *semi-explicit DAE* representation

$$\begin{aligned}\dot{x}(t) &= f(x(t),\ y(t),\ t)\\ 0 &= g^1(x(t),\ y(t),\ t)\\ 0 &= g^2(x(t),\ y(t),\ t)\\ 0 &= g^3(x(t),\ y(t),\ t)\\ &\vdots\end{aligned}$$

with g^1 solvable for y, $det\left(\frac{\partial}{\partial y} g^1\right) \neq 0$ and $\left\{\frac{d}{dt} g_i^k\right\} \subseteq \{g_i^{k-1}\},\ k>1$.

Mass matrix DAE

$$M\dot{x}(t) = f(x(t),\ t)$$

where *mass matrix* M is singular.

An equivalent explicit formulation:

$$\begin{bmatrix} M_x & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \dot{x} \\ 0 \end{bmatrix} = \begin{bmatrix} \hat{f}(x(t),\ y(t),\ t) \\ g(x(t),\ y(t),\ t) \end{bmatrix}$$

See [cui2020mass].

Linear time-invariant DAE

$$E\dot{x}(t) = Ax(t)$$

where matrix E is singular.

Indices

There are multiple, sometimes closely related, definitions of indices for DAE systems. Usually these indices are a measure of how far from an ODE the DAE is.

Differential Index

The minimum number of times a DAE

$$F(\dot{x}(t),\ x(t),\ t) = 0$$

has to be differentiated with respect to time t to be able to determine $\dot{x}(t)$ as a function of t and x is called the *differential index*.

See [hairer2006numerical].

Perturbation Index

The *perturbation index* measures the sensitivity of the solution of a DAE to perturbations of its right-hand side. The DAE

$$F(\dot{x}(t),\, x(t),\, t) = 0$$

has perturbation index p along a solution $x(t)$ if p is the smallest non-negative integer such that the solution $\tilde{x}(t)$ of the perturbed system

$$F(\dot{\tilde{x}}(t),\, \tilde{x}(t),\, t) = \delta(t)$$

satisfies

$$\| x(t) - \tilde{x}(t) \| \leq C\left(\| x(t_0) - \tilde{x}(t_0) \| + \max_{t_0 \leq \xi \leq t} \| \delta(\xi) \| + \max_{t_0 \leq \xi \leq t} \| \dot{\delta}(\xi) \| + \cdots + \max_{t_0 \leq \xi \leq t} \| \delta^{(p-1)}(\xi) \| \right)$$

for some constant C and a sufficiently small and smooth perturbation $\delta(t)$. A DAE with perturbation index greater than 1 is called a *higher-index DAE*.

See [hairer2006numerical].

Example of Structural Singularity

[cellier2006continuous, chapter 7.7 Structural Singularity Elimination] provides a simple electric circuit model that is a higher-index DAE.

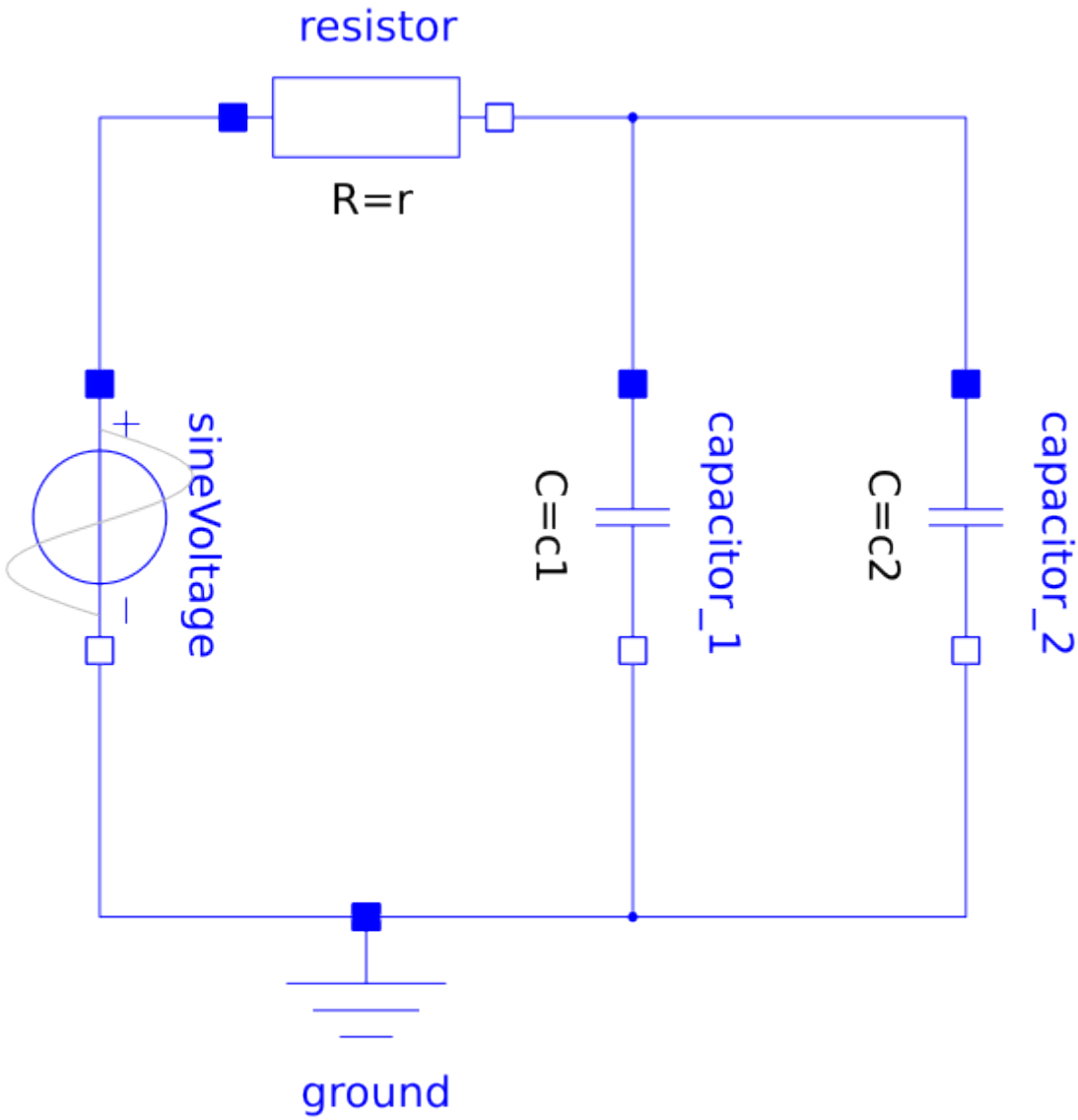


Figure 1. Modelica model of an electric circuit with two capacitors in parallel.

The circuit has two capacitors in parallel and can be represented with equations:

$$
\begin{aligned}
v_0 &= f(t) \\
v_R &= R \cdot i_0 \\
i_1 &= C_1 \cdot \dot{v}_1 \\
i_2 &= C_2 \cdot \dot{v}_2 \\
v_0 &= v_R + v_1 \\
v_2 &= v_1 \\
i_0 &= i_1 + i_2
\end{aligned}
$$

When both capacitive voltages v_1 and v_2 are chosen as state variables, equation $v_2 = v_1$ has no unknowns left, so it is a *constraint equation*.

There are different ways to solve this. The modeler could change the causality or exporting tools can try to reduce the perturbation index symbolically [2] to end up with a consistent initialization for the system.

Hello World DAE Example

A simple initial-value problem

$$\begin{aligned} \dot{x} &= y \\ y + 2z &= x \\ y - 3z &= 2x \end{aligned}$$

with variables $x, y, z \in \mathbb{R}$ and start condition

$$\begin{pmatrix} x(t_0) \\ y(t_0) \\ z(t_0) \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} \text{and } \dot{x}(t_0) = \dot{x}_0$$

at time t_0. For this specific example a start condition x_0 is sufficient to calculate consistent start values. The independent variable (usually time) t is omitted for simplicity.

Hello World DAE - Fully-Implicit DAE

The fully-implicit representation

$$F(\dot{x}, x, t) = 0$$

is given by

$$F : \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R} \to \mathbb{R}^3, \begin{pmatrix} x \\ \dot{x} \\ t \end{pmatrix} \mapsto \begin{pmatrix} \dot{x}_1 - x_2 \\ x_2 + 2x_3 - x_1 \\ x_2 - 3x_3 - 2x_1 \end{pmatrix}$$

Hello World DAE - Semi-Explicit DAE

The semi-explicit representation

$$\begin{aligned} \dot{x} &= f(x, y, t) \\ 0 &= g(x, y, t) \end{aligned}$$

is given by

$$\begin{aligned} f &: \mathbb{R} \times \mathbb{R}^2 \times \mathbb{R} \to \mathbb{R}, \ (x, y, t) \mapsto \begin{pmatrix} y_1 \end{pmatrix} \\ g &: \mathbb{R} \times \mathbb{R}^2 \times \mathbb{R} \to \mathbb{R}, \ (x, y, t) \mapsto \begin{pmatrix} y_1 + 2y_2 - x \\ y_1 - 3y_2 - 2x \end{pmatrix} \end{aligned}$$

Hello World DAE - Explicit ODE

This simple system could also be transformed into an explicit ODE formulation

$$\dot{x} = \frac{7}{5} x$$

with local variables y and z

$$y = \frac{7}{5}x$$
$$z = -\frac{x}{5}$$

and exported as an ODE ModelExchange FMU.

3. Use Cases

3.1. Generic DAE Use Case

There are a number of possible reasons to export a model as a system of differential-algebraic equations (DAE) instead of a system of ordinary differential equations (ODE):

- The exporting tool doesn't support methods to transform a DAE into ODE representation.
- The time needed to perform symbolic transformation to ODE representation takes a significant amount of time for large systems.
- Large system models are usually characterized by a high degree of sparsity. Some DAE solvers can utilize sparsity to reduce simulation time.
- More efficient simulation of models with large algebraic loops for DAE representation.

See [braun2017solving].

3.1.1. Modelica Example CauerLowPassAnalog

The Modelica model Modelica.Electrical.Analog.Examples.CauerLowPassAnalog from the Modelica Standard Library [3]

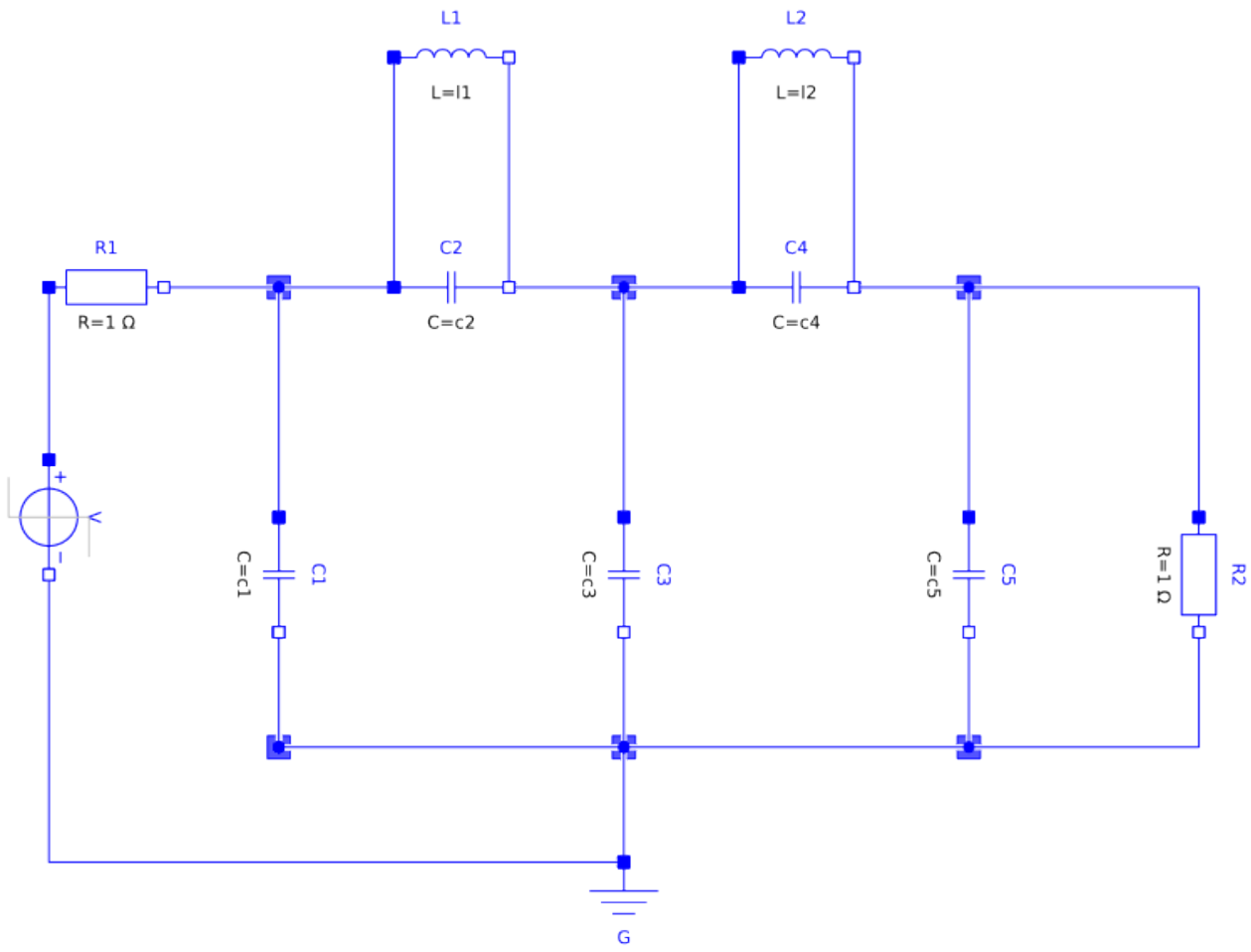


Figure 2. Graphical representation of the Cauer low-pass-filter `Modelica.Electrical.Analog.Examples.CauerLowPassAnalog`.

has structural singularities.

To produce a complete and consistent set of initial conditions for the DAE, exporting tools usually use a symbolic index reduction algorithm.

The model has nine state candidates

```
1: L1.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
2: L2.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
3: C2.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
4: C4.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
5: L1.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
6: L2.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
7: C1.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
```

```
8: C5.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
9: C3.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
```

and four constraint equations

```
1: L2.v = C3.v - C5.v
2: L1.v = C1.v - C3.v
3: C4.v = L2.v
4: C2.v = L1.v
```

Modelica tools that want to export an ODE ModelExchange FMU need to reduce the perturbation index to index 1 to end up with an ODE representation of the DAE system.

One option to achieve this is the so called dummy derivative method. From the set of state candidates, five state variables are chosen (e.g. `C1.v`, `C3.v`, `C5.v`, `L1.i`, `L2.i`) and the constraint equations

$$\begin{aligned} C2.v &= L1.v \\ L1.v &= C1.v - C3.v \\ C4.v &= L2.v \\ L2.v &= C3.v - C5.v \end{aligned}$$

are differentiated

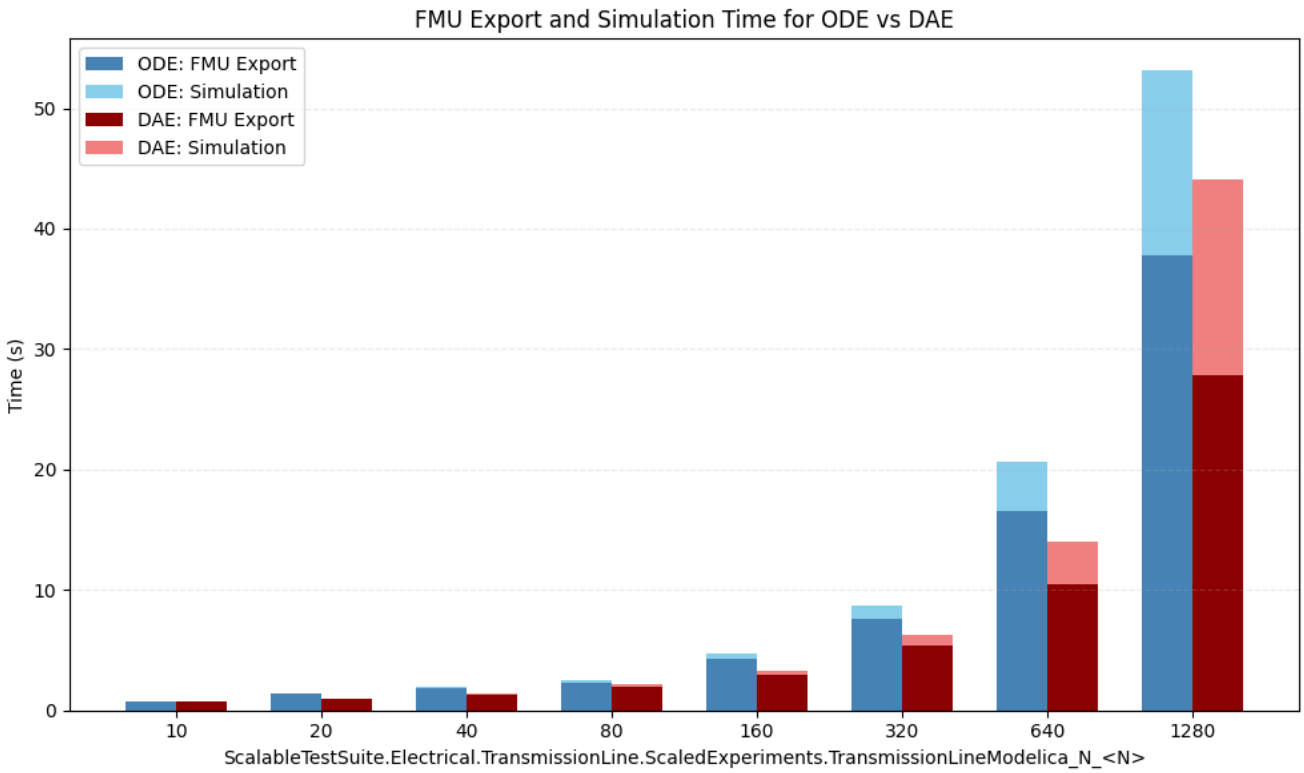
$$\begin{aligned} \frac{d}{dt}C2.v &= \frac{d}{dt}L1.v \\ \frac{d}{dt}L1.v &= \frac{d}{dt}C1.v - \frac{d}{dt}C3.v \\ \frac{d}{dt}C4.v &= \frac{d}{dt}L2.v \\ \frac{d}{dt}L2.v &= \frac{d}{dt}C3.v - \frac{d}{dt}C5.v \end{aligned}$$

which results in four additional dummy states (`$DER.L1.v`, `$DER.L2.v`, `$DER.C2.v`, `$DER.C4.v`). Here the Modelica keyword `stateSelect` influences how variables are picked.

3.1.2. Scaling Power Grids Example

For simulation models representing power grids on a national or larger scale, traditional ODE solvers may struggle with convergence and performance. Also, the time needed to perform symbolic transformation from DAE to ODE is growing with the number of equations.

See for example Modelica models `ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_10` to `ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_1280` from ScalableTestSuite [4].



The simulation was performed using `OpenModelica v1.27-dev-92` with the following settings:

- DAE export: `--daeMode`
- Method: Sundials IDA [5]

Since OpenModelica does not support FMI 3.0 or FMI-LS-DAE at the time of writing, the export and simulation times were measured using OpenModelica's default C export and simulation.

4. Layered Standard Manifest File

This layered standard requires the use of a layered standard manifest file and it shall be stored inside the FMU at the following path: `/extra/org.fmi-standard.fmi-ls-dae/fmi-ls-manifest.xml`.

[Add figure here] shows the root element `fmiLayeredStandardManifest`.

Two additional elements - `<AlgebraicVariables>` and `<ModelStructure>` - are included in the layered standard manifest file to describe the DAE formulation.

Table 2. `fmiModelDescription` *element details.*

Element	Description
`<AlgebraicVariables>`	List of all algebraic variables exposed for the DAE formulation.

Element	Description
`<ModelStructure>`	Defines the structure of the DAE formulation for the model. Especially, the ordered lists of `outputs`, continuous-time states, and the initial unknowns (the unknowns during [InitializationMode]) are **re**-defined and overwrite the corresponding `<ModelStructure>` present in the `modelDescription.xml`, with the addition of the `residuals` element. Furthermore, the dependency of the unknowns from the knowns (as described in [model-dependencies]) can be optionally defined for `outputs`, continuous-time states and initial unknowns. The algebraic variables can now also be included as dependencies.
[Annotations]	Optional annotations for the top-level element.

The XML attributes of `<fmiLayeredStandardManifest>` are:

Table 3. `<fmiLayeredStandardManifest>` attribute details.

Attribute	Namespace	Value	Description
`fmi-ls-name`	`http://fmi-standard.org/fmi-ls-manifest`	`org.fmi-standard.fmi-ls-dae`	Name of the layered standard in reverse domain name notation.
`fmi-ls-version`	`http://fmi-standard.org/fmi-ls-manifest`	`0.0.1`	Version of the layered standard. This layered standard uses semantic versioning, as defined in [PW13].
`fmi-ls-description`	`http://fmi-standard.org/fmi-ls-manifest`	`Layered standard for DAE support in FMI`	String with a brief description of the layered standard that is suitable for display to users.

4.1. Algebraic variables

The element `<AlgebraicVariables>` defines the list of the algebraic variables.

Table 4. AlgebraicVariables elements.

Element	Description
`AlgebraicVariable`	The list of all algebraic variables present in the `ModelVariables` element of the modelDescription.xml.

Each `[AlgebraicVariable]` has only one attribute defining the value reference.

Table 5. Attributes to [AlgebraicVariable] element.

Attribute	Description
`valueReference`	The value reference for the algebraic variable present in the `ModelVariables` of the modelDescription.xml.

4.2. Model Structure

The structure of the model for the DAE-formulation is defined in element `<ModelStructure>`. It defines the dependencies between variables. The `<ModelStructure>` element is extended with an additional element - `<Residual>` - and the optional dependencies can now include algebraic variables.

An FMU that follows this layered standard must expose all residuals with the `<Residual>`.

It should be noted that the ModelStructure below is not a straight extension of the ModelStructure from the core standard, since it e.g. does not specify how the elements `ClockedState` or `EventIndicator` are affected by the DAE-formulation. That is also why some elements in Table 6 are defined in a similar way to the core standard, with the exception that references to concepts in co-simulation, clocks, and events are excluded. Moreover, notes and examples that are already present in the core standard are omitted here.

Similar to the `ModelStructure` from the core FMI-standard, the optional part of the model structure defines in which way `derivatives`, `outputs`, and initial unknowns depend on `inputs` and/or `parameters`, and continuous-time states. Therefore the concepts from Model Exchange in the core-standard that are applicable to the DAE-formulation are repeated. The unknowns are extended with the `<Residual>` element, and each unknown can now also depend on `<AlgebraicVariables>`.

Table 6. ModelStructure elements.

Element	Description
`Output`	Ordered list of all outputs, in other words, a list of value references where every corresponding variable must have [causality] = [output] (and every variable with [causality] = [output] must be listed here). Attribute `dependencies` defines the dependencies of the `outputs` from the knowns after the [InitializationMode]. The functional dependency is defined as: $(\mathbf{y}_c, \mathbf{y}_d) := \mathbf{f}_{output}(\mathbf{x}_c, \mathbf{z}_c, \mathbf{u}_c, \mathbf{u}_d, t, \mathbf{p})$
`ContinuousStateDerivative`	Ordered list of value references of all derivatives of continuous states. The order defined by this list defines the order of the elements for [fmi3GetContinuousStates], [fmi3SetContinuousStates], and [fmi3GetContinuousStateDerivatives]. The corresponding continuous-time states are defined by attribute [derivative] of the corresponding variable state-derivative element. The functional dependency is defined as: $\dot{\mathbf{x}}_c := \mathbf{f}_{der}(\mathbf{x}_c, \mathbf{z}_c, \mathbf{u}_c, \mathbf{u}_d, t, \mathbf{p})$

Element	Description
`Residual`	Ordered list of residual equations (constraints). Each `<Residual>` contains one or more `<Formulation>` child elements. When a single `<Formulation>` is present, it defines the residual equation directly. When multiple `<Formulation>` elements are present within the same `<Residual>`, they represent the same constraint differentiated to successive orders. This grouping tells the the importer that the equations are structurally related, which can e.g. be used to perform index reduction with dummy derivatives or projection methods. Note that each formulation inside one residual element must be unique variables in the FMU. The functional dependency is defined as: $0 := \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{z}_c, \mathbf{u}_c, t, \mathbf{p})$
`InitialUnknown`	Ordered list of all exposed unknowns in [InitializationMode]. This list consists of all variables which: • are not clocked variables and have [causality] = [output] (with [initial] = [approx] or [calculated]), or • have [causality] = [calculatedParameter], or • are continuous-time states or state derivatives (defined with elements `<ContinuousStateDerivative>`) with [initial] = [approx] or [calculated], or • are algebraic variables that the FMU wishes to initialize in initialization mode. The resulting list is not allowed to have duplicates (for example, if a [state] is also an [output], it is included only once in the list). Attribute `dependencies` defines the dependencies of the unknowns from the knowns in [InitializationMode]. The functional dependency is defined as: $\mathbf{v}_{initialUnknowns} := \mathbf{f}_{init}(\mathbf{u}_c, \mathbf{u}_d, t_{start}, \mathbf{v}_{initial\,=\,exact})$ Since, `outputs`, continuous-time states and state derivatives are either present as knowns (if [initial] = [exact]) or as unknowns (if [initial] = [approx] or [calculated]), they can be inquired with `fmi3Get{VariableType}` in [InitializationMode].

Elements `<Output>`, `<ContinuousStateDerivative>`, and `<InitialUnknown>` have (partially) the following attributes. The `<Residual>` element differs in that its attributes are carried by its `<Formulation>` child elements, as described in Table 8.

Table 7. `<Output>`, `<ContinuousStateDerivative>`, and `<InitialUnknown>` common attributes details.

Attribute	Description
`valueReference`	The value reference of the unknown $v_{unknown}$.

Attribute	Description
`dependencies`	Optional attribute defining the algebraic dependencies as list of value references of the unknown $\mathbf{v}_{unknown}$ directly with respect to $\mathbf{v}_{known}$. For a real valued unknown and a real valued known, if the known is not listed among the dependencies then the partial derivative of the unknown with respect to that known is identically zero. If dependencies is not present, it must be assumed that the unknown depends on all knowns. If dependencies is present as empty list, the unknown depends on none of the knowns. Otherwise the unknown depends on the knowns defined by the given value references. Knowns $\mathbf{v}_{known}$ in [ContinuousTimeMode] (ME) for `<Output>` and `<ContinuousStateDerivative>` elements are: • inputs (variables with [causality] = [input]), • continuous states, • parameters (variables with [causality] = [parameter]), • algebraic variables (variables listed in the `<AlgebraicVariables>` element) • [independent] variable (usually time; [causality] = [independent]). Knowns $\mathbf{v}_{known}$ in [InitializationMode] (for elements `<InitialUnknown>`) are: • inputs (variables with [causality] = [input]), • variables with [initial] = [exact], • algebraic variables (variables listed in the `<AlgebraicVariables>` element) • [independent] variable (usually time; [causality] = [independent]).

Attribute	Description
`dependenciesKind`	If `dependenciesKind` is present, `dependencies` must be present and must have the same number of list elements. If `dependenciesKind` is not present, it must be assumed that the unknown $\mathbf{v}_{unknown}$ depends on the knowns $\mathbf{v}_{known}$ without a particular structure. Otherwise, the corresponding known $\mathbf{v}_{known,\,i}$ enters the equation as: = `dependent`: no particular structure, $\mathbf{f}(\ldots,\ \mathbf{v}_{known,\,i},\ \ldots)$ The following `dependenciesKind` is only allowed for floating point $\mathbf{v}_{unknown}$: = [constant]: constant factor, $\mathbf{c} \cdot \mathbf{v}_{known,\,i}$ where $\mathbf{c}$ is an expression that is evaluated before [fmi3EnterInitializationMode] is called. The following `dependenciesKind` is only allowed for floating point $\mathbf{v}_{unknown}$ in [ContinuousTimeMode] (ME), and not for `<InitialUnknown>` for [InitializationMode]: = [fixed]: fixed factor, $\mathbf{p} \cdot \mathbf{v}_{known,\,i}$ where $\mathbf{p}$ is an expression that is evaluated before [fmi3ExitInitializationMode] is called. = [tunable]: tunable factor, $\mathbf{p} \cdot \mathbf{v}_{known,\,i}$ where $\mathbf{p}$ is an expression that is evaluated before [fmi3ExitInitializationMode] is called and in [EventMode] or at a communication point (ME and CS) due to a change of a [tunable] parameter. = [discrete]: discrete factor, $\mathbf{d} \cdot \mathbf{v}_{known,\,i}$ where $\mathbf{d}$ is an expression that is evaluated before [fmi3ExitInitializationMode].

4.2.1. Formulation

Each `<Formulation>` element within a `<Residual>` has the following attributes:

Table 8. Attributes to `<Formulation>` element.

Attribute	Description
`valueReference`	The value reference of the residual variable $v_{unknown}$, which must be present in the `ModelVariables` element of the `modelDescription.xml`.
`index`	Optional attribute indicating the DAE index of the original constraint, i.e., how many times this particular equation must be differentiated in order to solve for a particular derivative.

Attribute	Description
`dependencies`	Optional attribute defining the algebraic dependencies as a list of value references of the knowns $\mathbf{v}_{known}$ that this formulation directly depends on. Knowns $\mathbf{v}_{known}$ in [ContinuousTimeMode] (ME) for `<Formulation>` elements are: • inputs (variables with [causality] = [input]), • continuous states, • continous state derivatives • parameters (variables with [causality] = [parameter]), • algebraic variables (variables listed in the `<AlgebraicVariables>` element), • [independent] variable (usually time; [causality] = [independent]). If `dependencies` is not present, it must be assumed that the formulation depends on all knowns. If `dependencies` is present as an empty list, the formulation depends on none of the knowns. Otherwise the formulation depends on the knowns defined by the given value references.
`dependenciesKind`	See the description of `dependenciesKind` in Table 7. TODO: Extend to help importer?

Example

Consider the following index 2 DAE system:

$r_1 = C_1 \frac{dx_1}{dt} + y = 0$

$r_2 = C_2 \frac{dx_2}{dt} - y + 1 = 0$

$r_3 = x_1 - x_2 = 0$

Assume it is exported as a DAE FMU with the following variables:

Variable	valueReference
x_1	1
x_2	2
y	3
$\frac{dx_1}{dt}$	4
$\frac{dx_2}{dt}$	5
r_1	6
r_2	7
r_3	8
$\frac{dr_3}{dt}$	9

Case 1: No structural analysis. The importer is responsible to figure out how to solve the system:

```
<ModelStructure>
  <Residual> <!-- r1 -->
    <Formulation
      valueReference="6"
      dependencies="3 4"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r2 -->
    <Formulation
      valueReference="7"
      dependencies="3 5"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r3 -->
    <Formulation
      valueReference="8"
      dependencies="1 2"
      dependenciesKind="dependent dependent"/>
  </Residual>
</ModelStructure>
```

Case 2: The FMU additionally indicates that r_3 is a constraint of index 2, i.e., it must be differentiated twice to obtain an index 1 system. This gives the importer structural information to know to apply a solver able to handle index 2 problems:

```
<ModelStructure>
  <Residual> <!-- r1 -->
    <Formulation
      valueReference="6"
      dependencies="3 4"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r2 -->
    <Formulation
      valueReference="7"
      dependencies="3 5"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r3, index=2 signals this constraint needs 2 differentiations -->
    <Formulation
      index="2"
      valueReference="8"
      dependencies="1 2"
      dependenciesKind="dependent dependent"/>
  </Residual>
</ModelStructure>
```

Case 3: The FMU exposes both r_3 and its time derivative $\dot{r}_3$ grouped within the same `<Residual>`, which indicates that these are the original constraint and its differentiated form. The importer can use this grouping to apply the dummy derivatives algorithm or use r_3 as a drift-correction projection when integrating the index 1 formulation:

$$r_3 = x_1 - x_2 = 0$$

$$\dot{r}_3 = \frac{dx_1}{dt} - \frac{dx_2}{dt} = 0$$

```
<ModelStructure>
  <Residual> <!-- r1 -->
    <Formulation
      valueReference="6"
      dependencies="3 4"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r2 -->
    <Formulation
      valueReference="7"
      dependencies="3 5"
      dependenciesKind="dependent dependent"/>
  </Residual>
  <Residual> <!-- r3 and its derivative, grouped as related formulations -->
    <Formulation <!-- r3 -->
      index="2"
      valueReference="8"
      dependencies="1 2"
      dependenciesKind="dependent dependent"/>
    <Formulation <!-- der(r3) -->
      index="1"
      valueReference="9"
      dependencies="4 5"
      dependenciesKind="dependent dependent"/>
  </Residual>
</ModelStructure>
```

The API for partial derivative, `GetDirectionalDerivative` and/or `GetAdjointDerivative` could be used with this structure to apply the dummy derivatives algorithm, or the index 2 constraint can serve as a projection constraint when integrating the index 1 formulation.

References

- [cui2020mass] Cui, Hantao, Fangxing Li, and Joe H. Chow. "Mass-matrix differential-algebraic equation formulation for transient stability simulation." arXiv preprint arXiv:2008.03883, 2020.
- [cellier2006continuous] Cellier, François E., and Ernesto Kofman. "Continuous system simulation". Boston, MA: Springer US, 2006.
- [hairer2006numerical] Hairer, Ernst, Christian Lubich, and Michel Roche. "The numerical

solution of differential-algebraic systems by Runge-Kutta methods". Vol. 1409. Springer, 2006.

- [cellier1993automated] Cellier, Francois E., and Hilding Elmqvist. "Automated formula manipulation supports object-oriented continuous-system modeling". IEEE Control Systems Magazine 13.2 (1993): 28-38.
- [braun2017solving] Willi Braun, Francesco Casella, and Bernhard Bachmann. "Solving Large-Scale Modelica Models: New Approaches and Experimental Results Using OpenModelica". *Linköping Electronic Conference Proceedings (Print)*, pp. 557-563. Linköping University Electronic Press, 2017. https://re.public.polimi.it/bitstream/11311/1065357/1/2017-BraunCasellaBachmann.pdf
- [casella2015simulation] Casella, Francesco. "Simulation of Large-Scale Models in Modelica: State of the Art and Future Perspectives". Proceedings of the 11th International Modelica Conference, pp. 459-468, 2015. https://dx.doi.org/10.3384/ecp15118459

[1] Source: https://en.wikipedia.org/wiki/Differential-algebraic_system_of_equations

[2] e.g. by using Pantelides algorithm, see [cellier1993automated]

[3] Modelica Standard Library version 4.1.0

[4] https://github.com/casella/ScalableTestSuite, see also [casella2015simulation]

[5] https://computing.llnl.gov/projects/sundials/ida